CHAPTER 1: CSE vs ML vs AI — Getting the Foundations Right

1.1 What is Computer Science (CSE)?

Definition:
Computer Science is the core study of computation, algorithms, hardware-software systems, and the theory behind programming. It's the scientific and practical approach to computation and its applications.

Key Areas in CSE:

- **Data Structures & Algorithms**: Efficient ways to store, access, and process data.
- **Operating Systems**: How computers manage hardware and software resources.
- **Computer Networks**: How data travels between systems over the internet or intranet.
- **Databases**: How data is stored, retrieved, and managed efficiently.
- **Software Engineering**: Designing, developing, testing, and maintaining software.
- **Computer Architecture**: Understanding processors, memory, and low-level machine interaction.

Real-World Example:
How does Instagram load so fast when you open it?
Behind the scenes:

- **Databases** fetch your profile and feed instantly.
- **Caching systems** (like Redis or Memcached) store recent data for fast access.
- **Load balancing and distributed systems** handle millions of users concurrently.
- **Efficient code and APIs** written in scalable programming languages.

Common Programming Languages in CSE:

- **Python** – Popular for scripting, backend, and ML.

- **C++** – Systems programming and performance-focused applications.
- **Java** – Enterprise applications, Android development.
- **JavaScript** – Web development (frontend & backend with Node.js).

Top Courses to Learn CSE:

- **CS50x by Harvard** (Free, beginner-friendly, covers fundamentals to advanced topics)
- **MIT OpenCourseWare** – Advanced and in-depth CS courses like Algorithms, OS, Systems.

1.2 What is Machine Learning (ML)?

Definition:
Machine Learning is a subset of AI where machines learn from data to make decisions or predictions **without being explicitly programmed for each rule**.

Core Idea:
Instead of writing rules manually (if X then Y), you provide data (input-output pairs), and the model **learns the mapping** automatically.

Types of Machine Learning:

1. **Supervised Learning**
    - Labeled data is used.
    - Example: Email spam detection (emails labeled “spam” or “not spam”).
    - Algorithms: Linear Regression, Decision Trees, SVM, Neural Networks.
2. **Unsupervised Learning**
    - No labels, the model finds structure in data.
    - Example: Grouping customers based on behavior (clustering).
    - Algorithms: K-Means, PCA, DBSCAN.
3. **Reinforcement Learning**
    - Learning through rewards and penalties (trial and error).

    - Example: Teaching a robot to walk or an AI to play games like Chess or Atari.
    - Concepts: Agent, Environment, Reward Function.

Real-World Example:
You show a model **10,000 images of cats**, labeled as "cat."
It **learns visual features** (ears, shape, patterns) and then predicts whether a new image contains a cat.

Tools and Libraries in ML:

- **Scikit-learn**
- **TensorFlow**
- **PyTorch**
- **Keras**

Top Courses to Learn ML:

- **Andrew Ng's ML Course** (Coursera, Stanford-based, beginner to intermediate level)
- **Fast.ai ML for Coders** (Hands-on, deep learning focused, great for coders)

1.3 What is Artificial Intelligence (AI)?

Definition:
Artificial Intelligence refers to the broader field where **machines simulate human-like intelligence** – not only learning patterns (ML), but also **thinking, reasoning, planning, and interacting**.

AI is a **superset of ML**. While ML is mainly data-driven learning, **AI combines ML with logic, memory, decision-making, and even sensory processing** (like vision, speech).

Key Subfields of AI:

- **Machine Learning** – Learning from data (covered above).
- **Symbolic AI / Expert Systems** – Rule-based systems used before ML boom.
- **Natural Language Processing (NLP)** – Understanding and generating human language.
- **Computer Vision** – Interpreting visual information (images, video).

- **Robotics** – Intelligent movement and manipulation of physical objects.
- **Planning & Reasoning** – Decision-making processes, logical inference.
- **Knowledge Representation** – Storing knowledge in a way machines can reason about.

Real-World Example:
♟️ A chess-playing AI doesn't just recognize past game patterns (ML), it also:

- **Plans** several moves ahead.
- **Evaluates** the game state.
- **Remembers** previous strategies.
- **Adapts** to new opponents.

Hence, it uses **AI = ML + Planning + Reasoning + Memory + Sensory Input**

AI Applications:

- Chatbots (like ChatGPT)
- Self-driving cars
- Fraud detection
- Language translation
- Virtual assistants (like Siri, Alexa)

CHAPTER 2: AI vs Generative AI — Machines That Think vs Create

2.1 Traditional AI

What is Traditional AI?

Traditional AI refers to systems built to **simulate intelligence through logic, rules, and statistical models**. These systems are **not creative** — they are designed to **analyze, classify, and make decisions** based on data patterns.

Key Characteristics:

- **Rules-based systems**: If-else logic, expert systems.
- **Decision trees & regression models**: Predictive but not generative.
- **Goal**: Accuracy in prediction, not content generation.

What Traditional AI Does:

- Detect spam in emails
- Recommend products on Amazon
- Predict if a loan will default
- Flag fraudulent transactions

Real-World Examples:

- **Fraud Detection System**: Flags suspicious bank transactions based on historical data.
- **Spam Filter**: Classifies emails as spam or not spam.
- **Recommendation Engine**: Suggests YouTube videos or Netflix shows based on your past behavior.

Top Resources to Learn Traditional AI:

1. **Artificial Intelligence by Georgia Tech (Udacity)**
   (Intermediate Level, focuses on classical AI: search, logic, planning, and decision-making)
2. **Stanford CS221: AI - Principles & Techniques**
   (University course — advanced, includes logic, search, planning, and probabilistic reasoning)
3. **Edureka's YouTube Playlist – AI for Beginners**

   (Free visual walkthrough of traditional AI concepts like search algorithms, decision trees, expert systems)

2.2 Generative AI (GenAI)

What is Generative AI?

Generative AI refers to **AI systems that create new content**, such as text, images, audio, video, and code, by learning from massive datasets. These models go **beyond decision-making** and are capable of **creative tasks**.

How Generative AI Works:

- Trained on large datasets (books, images, code, music).
- Learns patterns and **probabilities** of what comes next.
- Generates **new** outputs that resemble the training data, but aren't direct copies.

Types of Generative AI:

Type	Tools/Models
Text	GPT-4, Claude, Gemini
Images	DALL·E, Midjourney, Stable Diffusion
Audio	ElevenLabs, Suno
Video	Runway, Pika

Real-World Example:

Prompt: "Create a poem about time in Shakespearean style"
GPT-4 Output: A completely original, stylistically accurate Shakespearean poem about time.

Top Resources to Learn Generative AI:

1. **DeepLearning.AI: Generative AI Specialization**

   (Beginner-friendly, explains text/image generation, prompt engineering, and fine-tuning)

2. **OpenAI Cookbook on GitHub**
   (Practical code examples on using GPT, embeddings, tools, function calling, etc.)

3. **YouTube - Fireship: What is Generative AI?**

   (Full course on generative AI by freecodecamp explains GenAI with clear visuals and real-world demos)

4. **Hugging Face Course**
   (Build and deploy GenAI models like transformers, language models, and diffusion)

CHAPTER 3: AI Model vs AI Tool — Brains vs Applications

3.1 What is an AI Model?

Definition:

An **AI model** is a **mathematical or computational algorithm** trained on data to make decisions, predictions, or generate content. It represents the **core logic** or **"brain"** behind intelligent behavior.

Think of it as:

- A trained system that **learns from past data**.
- Given **an input**, it produces a **predicted or generated output**.

Examples:

- **GPT-4**: A large language model (LLM) trained on massive internet text to generate human-like responses.
- **ResNet**: A convolutional neural network for image recognition tasks.
- **BERT**: A transformer model used for understanding the context of words in text.
- **Custom ML Model**: A logistic regression model trained to predict customer churn.

Key Features of AI Models:

- Require **training on large datasets**.

- Can be **fine-tuned** for specific tasks (e.g., domain-specific chatbots).
- Can be hosted and accessed via **APIs** (e.g., OpenAI's API).

Top Resources to Learn About AI Models:

1. **Hugging Face – Transformers Course**
   Covers how transformer-based models like BERT, GPT, and T5 work. Great for text models.
2. **Coursera – Deep Learning Specialization by Andrew Ng**
   In-depth courses on neural networks, CNNs, RNNs, and model building.
3. **YouTube – What is a Machine Learning Model (StatQuest)**

   Visual, beginner-friendly explanation on what ML models are and how they work.

3.2 What is an AI Tool?

Definition:

An **AI Tool** is a **software product or application** that **uses one or more AI models under the hood** to provide value to end-users. It often includes:

- A **user interface**
- **Backend APIs**
- **Storage, analytics**, or additional features

Think of it as:

- The **"productized" version** of AI models.
- A tool that makes complex AI models **usable for non-technical users**.

Examples:

Tool	Built On	Purpose
ChatGPT	GPT-4	Conversational AI
Midjourney	Diffusion Models	AI Image Generation
Jasper AI	GPT-3.5 / GPT-4	AI Writing Assistant

Copy.ai	LLMs (GPT)	Marketing content generation
Notion AI	GPT-based + APIs	Productivity and writing aid

Components in an AI Tool:

- **Frontend/UI**: To input prompts and view results.
- **Model Layer**: Interacts with one or more AI models.
- **Additional logic**: Workflow handling, file uploads, analytics, etc.

Top Resources to Learn About AI Tools:

1. **OpenAI Cookbook: Building with AI APIs**
   Real code examples to integrate GPT-3/4 into your apps.

2. **Build AI-Powered Tools with LangChain (YouTube – Prompt Engineering Guide)**
   Learn how to build actual tools with memory, reasoning, and chaining APIs.

3. **Google Vertex AI Tools**
   For those interested in building enterprise-grade tools with hosted models.

3.3 Key Differences Between AI Models & AI Tools

Feature	AI Model	AI Tool
Definition	Trained algorithm that makes predictions	Product using one or more AI models
Purpose	Core logic or intelligence engine	End-user-facing app for productivity, tasks
User Level	Developers, researchers	General public, creators, teams
Output	Raw predictions (e.g., text, label, score)	Refined interface or end result
Examples	GPT-4, BERT, DALL·E	ChatGPT, Jasper, Copy.ai, Runway
Training Needed?	Yes – trained on large datasets	No – tools are plug-and-play
Access Method	APIs, SDKs, inference libraries	Web apps, mobile apps, SaaS platforms

Analogy:

- **AI Model = Engine**
- **AI Tool = Car (with steering, GPS, AC, dashboard using the engine)**

Resources to Compare AI Tools vs Models:

1. **Understanding AI and Chat GPT**
   Visual breakdown of tools like ChatGPT vs underlying models like GPT-4.

2. **Medium Article: Everything about Ai Models**

   A beginner-friendly read on the conceptual understanding of models.

3. **LangChain Docs**
   How developers combine multiple models, memory, and logic to build full tools.

CHAPTER 4: Large Language Models (LLMs)

What is a Large Language Model (LLM)?

A **Large Language Model (LLM)** is a type of deep learning model, typically based on transformer architecture, trained on massive datasets containing natural language and sometimes other modalities (code, images, etc.). These models are capable of generating human-like responses by learning the statistical patterns in text.

LLMs are **pre-trained** on large corpora (web data, books, codebases, forums, etc.) and fine-tuned (or prompted) to perform downstream tasks such as:

- Text generation
- Summarization
- Translation
- Code generation
- Question answering
- Sentiment analysis
- Semantic search

- And more...

Key Examples of LLMs:

Model	Developer	Notable Features
GPT-4	OpenAI	Multimodal (text + image), used in ChatGPT
Claude	Anthropic	Constitutional AI approach, long context window
Gemini	Google DeepMind	Formerly Bard; focuses on reasoning + grounding
LLaMA	Meta	Open-source weights, research & community driven
Mistral	Mistral.ai	Small, efficient, open-weight performant models
Falcon	TII (UAE)	Open-weight model optimized for performance

How Do LLMs Work?

Architecture: Transformer-based Neural Networks

Most LLMs rely on the **Transformer architecture**, first introduced by Vaswani et al. in the 2017 paper *"Attention is All You Need."* The architecture uses self-attention mechanisms to process input in parallel, enabling much greater scalability compared to RNNs or LSTMs.

Training Objective: Language Modeling

LLMs are trained using one of the following:

- **Causal Language Modeling (CLM)**: Predict the next word/token given previous ones. Used in GPT-style models.

- **Masked Language Modeling (MLM)**: Predict masked tokens in a sequence. Used in BERT-style models.

Data Scale:

- GPT-3 was trained on **500B+ tokens**
- GPT-4's data is undisclosed but assumed to be **trillions of tokens**
- Models require **massive compute** (10K+ GPUs for months), **high-quality data**, and **careful alignment** to reduce bias/toxicity.

What Can LLMs Actually Do?

Task	Description
Text Generation	Compose essays, blogs, scripts, documentation
Summarization	Convert long articles or transcripts into bullet-point summaries
Translation	Translate across multiple languages with high fluency
Programming	Generate and autocomplete code (e.g., GitHub Copilot, Replit Ghostwriter)
Semantic Search	Return relevant documents based on meaning, not keywords
Reasoning & Logic	Solve math, puzzles, or multi-step logic tasks
Conversational AI	Used in chatbots, personal assistants, and therapy bots

Tool Use & Planning	Integrate with APIs to complete real-world tasks

Limitations of LLMs

Despite their intelligence, LLMs have several known drawbacks:

Limitation	Description
Lack of true understanding	LLMs don’t "understand" like humans—they predict based on patterns
Statelessness	Default LLMs don’t retain memory across sessions unless engineered in
Hallucination	May generate convincing but false or misleading information
Outdated Knowledge	Limited to data it was trained on; doesn’t know recent or real-time info
Context Window Limit	Can only consider a fixed amount of text (~8K to 200K tokens depending on model)

Real-World Applications of LLMs

Domain	Application Example	LLM’s Role

Email	Gmail Smart Compose	Predicts next phrase based on context
Software Dev	GitHub Copilot	Suggests code completions and refactors
Legal	Ironclad, Spellbook	Contract summarization and clause extraction
Healthcare	AI therapy apps like Woebot, Replika	Simulated therapy chats
Customer Support	AI chatbots on websites	Handles FAQs, product info, and simple troubleshooting

Building Your Own LLM (Realistically)

You likely won't train GPT-4 from scratch (costs ~$100M+), but you **can build powerful custom LLMs** by fine-tuning or instruct-tuning open-source models.

Options for Building:

Method	Description
Open-source models	Start with LLaMA 3, Mistral, or Falcon (weights are public)
Hugging Face Transformers	Most popular open-source NLP framework for model training/inference
Fine-tuning	Modify model behavior by training on custom data
LoRA / QLoRA	Lightweight, low-cost fine-tuning methods using adapters

Creating LLM using Python

Next Step: AI Agents (LLM + Memory + Tools + Autonomy)

Basic LLMs are reactive — they respond to prompts. **AI agents** are proactive — they **reason**, **plan**, and **act**.

What are AI Agents?

An **AI agent** is a system built on top of an LLM that has access to tools (APIs, search, file systems), memory, and logic modules to **autonomously achieve goals**.

Anatomy of an AI Agent:

- **LLM core**: Language reasoning & generation
- **Planner**: Determines sequence of actions
- **Tools/API access**: Executes external operations (e.g., search, booking)
- **Memory**: Maintains state, context across steps
- **Feedback loop**: Evaluates outcomes, refines next steps

Example:

Prompt: “Find 5 cheapest flights to Bali next month and book the best one.”

Component	Function
LLM	Interprets request and breaks it into steps
Tool Access	Calls flight APIs, web scrapers
Planning Logic	Compares prices, times, airlines
Execution Agent	Books the selected flight through the right portal

Memory	Remembers preferences and constraints

This is the foundation of **Auto-GPT**, **AgentGPT**, **OpenAgents**, and similar frameworks.

The Next Big Thing: RAG (Retrieval-Augmented Generation)

Why Do We Need RAG?

LLMs like GPT-4:

- **Don't know your private/company data**
- **Can't read PDFs or databases**
- **Forget context easily**
- **Hallucinate confidently**

RAG solves these problems.

RAG = Retrieval-Augmented Generation

Concept: Feed the LLM with **relevant, external knowledge at query time**.

How RAG Works:

1. **Document Ingestion**: PDFs, Notion docs, policies, reports
2. **Chunking**: Split text into semantically meaningful segments
3. **Embedding**: Convert chunks into vectors using an embedding model
4. **Vector DB Storage**: Store vectors in a searchable vector database
5. **Retrieval**: When prompted, retrieve top-K similar chunks
6. **Generation**: LLM responds using retrieved context

Real-World Use Case:

Query: "What's the refund policy in our 200-page HR PDF?"

- GPT-4 (alone): Doesn't know
- RAG-enabled system: Retrieves "refund policy" chunk → GPT-4 gives a precise answer

Tools & Stack for RAG

Component	Tools / Libraries
Chunking	LangChain, LlamaIndex, Haystack
Embedding	OpenAI, HuggingFace, Cohere, BAAI
Vector DB	Pinecone, Chroma, Weaviate, Qdrant, FAISS
LLM	GPT-4, Claude, Mistral, LLaMA
Serving APIs	FastAPI, LangServe, Flask

ChatGPT vs Full Custom RAG

Feature	ChatGPT Upload	Full RAG System
File Upload	Yes	Yes
Custom Chunking	No	Fully configurable
Vector Store Access	No	Full control

Multi-doc Scaling	Limited	Optimized for 1,000s of files
Persistent Knowledge	No	Yes (long-term memory)
Integration & Deployment	Not customizable	Fully deployable as chatbot/API/web app

Verdict: ChatGPT = Smart Assistant
RAG System = Enterprise-Grade Knowledge Engine

What is LangChain?

LangChain is an open-source Python framework that lets you build complex applications using LLMs + memory + tools.

Modules in LangChain:

Block Type	Function
LLM Block	Sends prompt to GPT-4, Claude, Mistral
Tool Block	Allows LLM to call calculator, search, APIs, SQL, etc.
Memory Block	Stores user history and context
Chain Block	Executes multi-step workflows
RAG Block	Enables doc retrieval + response generation

LangFlow – No-Code LangChain Builder

LangFlow is a visual GUI for LangChain. Think of it as **Canva for AI agents**.

- Drag-and-drop interface
- Connect LLM → Tools → Memory → Output nodes
- Deploy as API or UI app

What is a Vector Database?

LLMs don't understand keywords — they understand **semantic meaning**. A **vector database** allows similarity search on **semantic embeddings**.

Example:

- "How do I get a refund?" = [0.21, 0.88, 0.09…]
- "Cancel and get my money back" → similar vector → matched!

Popular Vector DBs:

Tool	Description
Pinecone	Scalable, fully managed cloud-native DB
Weaviate	Open-source, with modular plugins
ChromaDB	Python-native, great for prototyping
Qdrant	Fast, open-source with REST & gRPC support

[How i build AI Teacher with Vector Database]

Multi-Agent Control Panel (MCPs)

As AI agents evolve, you'll need to **coordinate multiple agents** for complex workflows.

What is MCP?

MCP (Multi-Agent Control Panel) is a **dashboard + orchestration system** to manage:

- Specialized agents (planner, researcher, executor)
- Task assignment and monitoring
- Communication between agents
- Dynamic feedback and control

Top Resources to Learn About AI Agents:

1. **LangChain + Agents Documentation**
   LangChain Agents
   Build agents that think and act using memory, tools, and LLMs.

2. **YouTube – How AutoGPT Works**
   AutoGPT Video

   A short, explanation of AI agents that plan and do.

3. **Blog: ReAct Prompting – Reason + Act**
   Original ReAct Paper

   How to guide LLMs with structured thinking and actions.

CHAPTER 5: Retrieval-Augmented Generation (RAG)

5.1 Why LLMs Alone Aren't Enough

Limitations of Standalone Large Language Models (LLMs)

While LLMs like GPT-4, Claude, or Mistral are trained on vast datasets, they have inherent limitations:

- **Static Knowledge**: LLMs possess knowledge only up to their training cut-off date and cannot access real-time information.
- **Hallucinations**: They may generate plausible-sounding but incorrect or fabricated information.

- **Token Limits**: There's a maximum limit to the amount of text they can process in a single prompt (e.g., GPT-4's 128K tokens).

- **No Personal Context**: LLMs lack awareness of specific user data, such as personal documents or proprietary company information.

Analogy

Consider asking a knowledgeable individual to analyze your company's latest sales report. If they've never seen the report and only have outdated business knowledge, their insights would be limited. Similarly, an LLM without access to current or specific data can't provide fully informed responses.

5.2 What is Retrieval-Augmented Generation (RAG)?

Definition

Retrieval-Augmented Generation (RAG) is an AI framework that enhances LLMs by integrating an information retrieval component. This allows the model to fetch relevant data from external sources, such as documents or databases, and incorporate it into its responses, thereby grounding its outputs in factual and up-to-date information. (Wikipedia)

Why RAG Works

- **Enhanced Accuracy**: By accessing external data, RAG reduces the chances of hallucinations, ensuring responses are based on actual information.

- **Real-Time Relevance**: It allows AI systems to provide answers that reflect the most current data available.

- **Contextual Responses**: RAG enables LLMs to tailor outputs based on specific user contexts or proprietary information.

RAG Workflow: Step-by-Step Breakdown

1. **Embedding**: Documents are divided into chunks and transformed into vector representations using embedding models (e.g., OpenAI's embeddings).

2. **Storage**: These vectors are stored in a vector database like Pinecone, FAISS, or Weaviate.

3. **Retrieval**: Upon receiving a query, the system retrieves the most relevant document chunks by comparing vector similarities.

4. **Generation**: The LLM processes the retrieved information alongside the original query to generate a comprehensive and accurate response.

Analogy

Imagine an AI chef (LLM) preparing a dish (response). Without RAG, the chef relies solely on memory. With RAG, the chef accesses fresh ingredients (retrieved data) from a well-stocked pantry (external sources), resulting in a more flavorful and accurate dish.

5.3 Practical Use Case of RAG

Scenario: Summarizing an HR Policy Document

- **Without RAG**: The LLM might provide a generic overview of HR policies, potentially missing specific details or including inaccuracies.
- **With RAG**: The system retrieves the actual HR policy document, extracts pertinent sections, and generates a precise summary, ensuring accuracy and relevance.(WIRED)

5.4 Tools and Technologies in the RAG Ecosystem

Core Components

- **Embedding Models**: Convert text into vector representations. Examples include OpenAI Embeddings, Cohere, and Hugging Face models.
- **Vector Databases**: Store and facilitate efficient retrieval of vectorized data. Options include Pinecone, FAISS, Weaviate, and ChromaDB.
- **Retrieval Frameworks**: Manage the retrieval process. Notable tools are LangChain, LlamaIndex, and Haystack.
- **LLMs (Generators)**: Generate human-like text based on inputs. Examples are GPT-4, Claude, Mixtral, and Gemini.
- **Orchestration Tools**: Coordinate the various components of the RAG pipeline. LangChain and RAGFlow are prominent examples.
- **Frontend/UI**: Provide user interfaces for interaction. Streamlit, Gradio, and custom applications are commonly used.

Additional Enhancements

- **PDF Parsing**: Tools like PyMuPDF and LangChain's PDFLoader facilitate the ingestion of PDF documents.
- **Web Scraping**: Libraries such as BeautifulSoup and Scrapy enable real-time data extraction from websites.

- **API Integration**: Incorporate live data (e.g., weather, stock prices) using RapidAPI or SerpAPI.

- **Fine-Tuning & Prompt Engineering**: Enhance LLM performance through customized prompts and memory chains.

5.5 ChatGPT and RAG Capabilities

ChatGPT Version	RAG Capability
Default ChatGPT	No retrieval; relies solely on pre-trained data.
ChatGPT with Browsing	Can access and retrieve real-time web information.
ChatGPT with File Uploads	Able to retrieve and process information from uploaded documents.
Custom GPTs	Utilize specific instructions and contextual data provided by the user.
API with Vector Store	Full RAG implementation achievable using tools like LangChain and Pinecone.

5.6 Resources to Master RAG

- **DeepLearning.AI: Retrieval-Augmented Generation Course**: Offers foundational knowledge and practical labs using OpenAI, Pinecone, and LangChain.

- **LangChain RAG Quickstart**: Provides a comprehensive tutorial for building RAG-powered applications.

- **LlamaIndex RAG YouTube Walkthrough**: An accessible guide for ingesting data and creating custom LLM interfaces.

- **OpenAI Cookbook – RAG Notebooks**: Features Jupyter notebooks demonstrating real-world RAG implementations with GPT.

Got it! Here's a concise **Resources to Learn RAG** section with brief descriptions and direct links:

Resources to Learn Retrieval-Augmented Generation (RAG)

1. **DeepLearning.AI RAG Course**
   Beginner-friendly course by Andrew Ng explaining RAG fundamentals, vector search, and building RAG applications.
   deeplearning.ai RAG Course

2. **LangChain Documentation**
   Official docs and tutorials for building RAG pipelines with vector stores and LLMs using LangChain framework.
   LangChain Docs

3. **OpenAI Cookbook – RAG Examples**
   Code examples and Jupyter notebooks demonstrating RAG workflows with OpenAI models and vector databases.
   OpenAI Cookbook on GitHub

4. **Pinecone Vector Database Docs**
   Documentation on how to store and query vector embeddings for efficient similarity search in RAG systems.
   Pinecone Docs

5. **LlamaIndex Tutorials**
   Tools and tutorials to connect LLMs with external knowledge sources, simplifying RAG implementation.
   LlamaIndex GitHub

6. **YouTube Tutorials**

- LangChain RAG full tutorial
- LlamaIndex basics Everything you need to Know

- Learning RAG from Scratch
- Master RAG in 5 Hours